What Claude needs in every session

The files Claude reads in every session, and your own

Florian Gärtner

Where we are today

	What Day 1 taught
→	**What Claude needs in every session** ← you are here
	What Claude needs only sometimes
	Your own skill
	Lunch
	A history you can go back to
	Where does this file live?
	Commit first, then let Claude move things. Then GitHub
	Wrap, done at 17:00

Learning objectives

After this block you can:

1. **Ask** Claude to write or improve your `CLAUDE.md` files, the project's and your global one, and **decide** what stays.
2. **Check** that a file works, by asking Claude in a new session what the file should tell it.
3. **Decide** which lines Claude needs in every session, and which file each goes in: your global `CLAUDE.md`, the project's, or `CLAUDE.local.md`.

From what Claude needs to a tested file

What Claude needs
about you and the project
in every session

→

`CLAUDE.md`
Claude drafts the lines
you approve them

→

Every session
reads it at the start
a request, not a lock

→

A new session
the same questions
did the answer change?

A file works when a new session gets the answers right.

The files that are always loaded

Which notes belong in `CLAUDE.md`?

Your `CLAUDE.md` files, once read, are handed to Claude with every turn. They come at four levels, from the widest to the narrowest.

- **Your global one**, `~/.claude/CLAUDE.md`, is read in every session, in every folder. It holds how you work and who you are.
- **The project's**, `CLAUDE.md` in the project's folder, is read in every session started there, and travels with the project to your coauthors. It holds what Claude needs to know about the project, in every session.
- **Your own for the project**, `CLAUDE.local.md` beside it, is read in the same sessions, but stays on your laptop. It holds what is true only on your laptop, or only for you.
- **A subfolder's**, `CLAUDE.md` in a subfolder, is read once Claude reads a file there. It holds what is true for that part of the project only.

Each line goes in the widest file where it is true.

Rules folders, and the check

One long `CLAUDE.md` gets hard to read and to keep current. A `rules/` folder splits its instructions into files by topic, each short enough to check. Your kit does this: `~/.claude/rules/rules.md` holds its conventions for files, data and code.

A rule file can also name the files it is for (`paths:` at its top). Then it is read only once Claude works on such a file, and takes no room otherwise. Here, `rules/r_code.md` is read only when Claude opens an R script.

```
---
paths:
  - "**/*.R"
---
- Fixed effects with `fixest`.
- Pipes: `|>`, not `%>%`.
```

Without that line, a rule file is read like the `CLAUDE.md` beside it: `~/.claude/rules/` in every session, a project's `.claude/rules/` in every session started there.

Claude gets all of these files together. None replaces another. A `/cd` into a project loads its files as well.

`/context all` lists the files loaded at the start.

What goes in them

What Claude needs to know in every session, and you would otherwise explain again.

```
# Survey project
- `income` is in thousands of euros.
- Variable names: snake_case.
- Raw data in `data/raw/` is read-only. Never write there.
- Scripts run in numbered order: 01_clean.R, 02_models.R.
```

After you correct Claude, your kit has Claude propose a line that would prevent it, and the file it belongs in. You decide whether it goes in.

Short, and not a guarantee

Every line in every one of these files takes up room in every turn.

- Keep each file under about **200 lines**. Longer files are followed less reliably.
- The docs' test for a line: *"Would removing this cause Claude to make mistakes?"* If not, the line goes.
- Lines you could check work better: *"After any change to `01_clean.R`, run `02_models.R` again"*, not *"Keep the scripts consistent"*.

> ⚠ **A request, not a lock**
>
> The docs call these files *"context, not enforced configuration"*. Claude usually follows them, but nothing forces Claude to.

A line works better with its reason

```
less effective   NEVER use ellipses

more effective   Your response will be read aloud by a text-to-speech
                 engine, so never use ellipses since the text-to-speech
                 engine will not know how to pronounce them.
```

“Claude is smart enough to generalize from the explanation.”

Anthropic, “Prompting best practices”, § Add context to improve performance. Checked 2026-10-05.

Say why. Claude can then apply the rule to cases you did not list.

Your turn

Pick a project Claude may read

Exercise

⚠ Every file Claude reads goes to the model provider.

Keep going in **the copy you worked in during "Try it out" (block 1_3)**. Its files may all go there.

A different project works too:

- **as it is**, if every file in it may go there
- **as a copy without the data**, and without your coauthors' drafts, if you are unsure

Time for your own files

Exercise

This slot is for your always-loaded files. **It is a menu: pick what is worth doing for you now. Nobody needs to do all of it.**

- **No `CLAUDE.md` in the project yet?** Have Claude write one.
- **Your global files?** With the kit they are `CLAUDE.md`, `rules/rules.md` and `about-me.md` in `~/.claude`. Have Claude change what does not fit you.
- **Files you already have?** Have Claude go over them.

Whichever you pick, the next slide is how to check it. A good file holds only what Claude needs in every session, and nothing you would not show a coauthor.

You ask. Claude writes. You read it and decide.

Ask, fix, ask again

Exercise

1. Ask Claude three things the file should tell it. For the project: who may change what, what must not be touched and why, how you want something done that your files do not show. For your global files: something true in all your projects. Note what Claude gets wrong. All right? Then ask about something your files do not show.
2. Tell Claude the right answers, then:

   > Put what you need to know to answer these correctly into the right file. Keep it short.

3. Read every line. Tell Claude what is wrong, and what it does not need in every session.
4. Type `/exit`, start `claude` again (for a global file, in a different folder), and ask the same questions. The file is read when a session starts, not when you change it.

A second opinion on your files

Exercise

For step 3, or for files you already have, Claude can check them against Anthropic's advice:

> Read my CLAUDE.md files and Anthropic's current advice on writing them, at code.claude.com/docs/en/best-practices. What should I cut, change or add? What could you read only when it is needed, instead of in every session?

- `/doctor prompt-audit` checks your instruction files for lines written for older models, files or commands that no longer exist, and rules that contradict each other. It proposes edits. Nothing changes until you say so.
- `/init`, with your file in place, suggests improvements instead of overwriting it.

Decide each change yourself.

If an answer did not change

Exercise

What you see	What to do
the file is not under **Memory files** in `/context all`	start Claude in the folder that holds the file
the file is listed, the answer is still wrong	turn the line into a concrete instruction
the file changed during the session	start a new session

Discussion

1. Share one line that changed an answer, and one that did not.
2. Claude suggested a line you cut, or a cut you refused. Who was right, and how would you find out?
3. Pick a line you were unsure about. Does it belong in your global file, the project's, or `CLAUDE.local.md`?
4. Your supervisor's prompt template is full of CRITICAL and MUST, with no reasons given. Would you rewrite it, and how would you find out whether it matters?

What to take away

1. Your `CLAUDE.md` files hold what Claude needs to know about the project, in every session.
2. What is true for all your projects goes in your global `CLAUDE.md`. What is true for one project goes in that project's.
3. Every line in every one of these files takes up room in every turn. Keep them short.
4. Say why. Claude can then apply the rule to cases you did not list.